

PyTorch with Slurm and clusters

From a training script to a job on a GPU

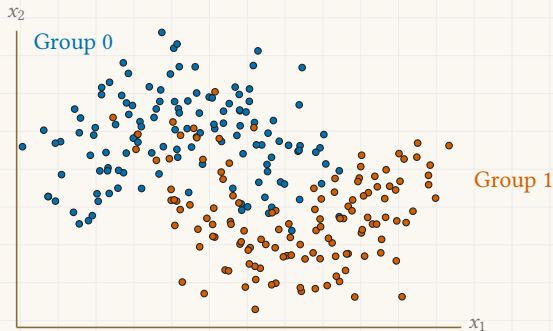
Carlos Cotrini

ETH Zurich

6 October 2026

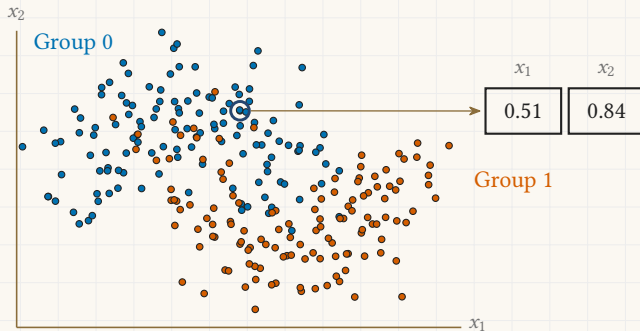
Two crescents, one network

Each point has two coordinates and belongs to one of two groups. The rest of the hour shows how to train a network that tells them apart, on a GPU of Santis.



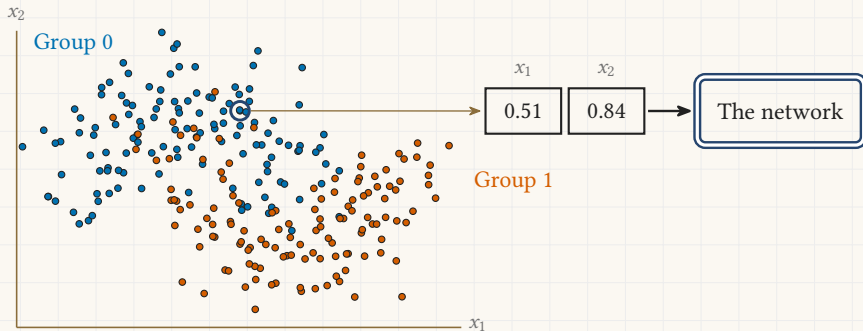
Two crescents, one network

Each point has two coordinates and belongs to one of two groups. The rest of the hour shows how to train a network that tells them apart, on a GPU of Santis.



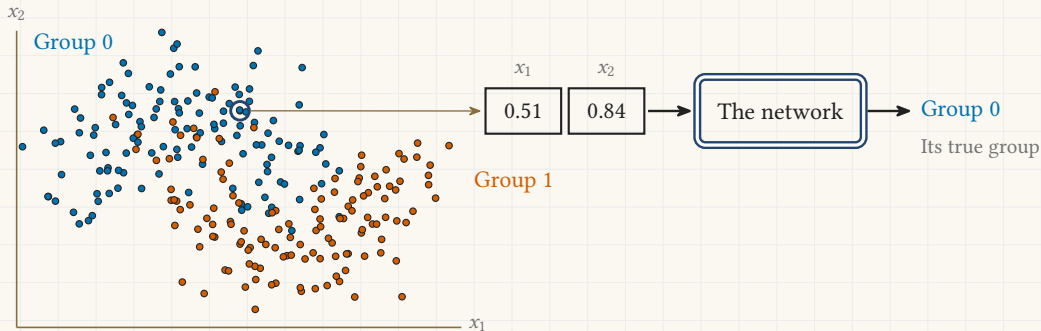
Two crescents, one network

Each point has two coordinates and belongs to one of two groups. The rest of the hour shows how to train a network that tells them apart, on a GPU of Santis.



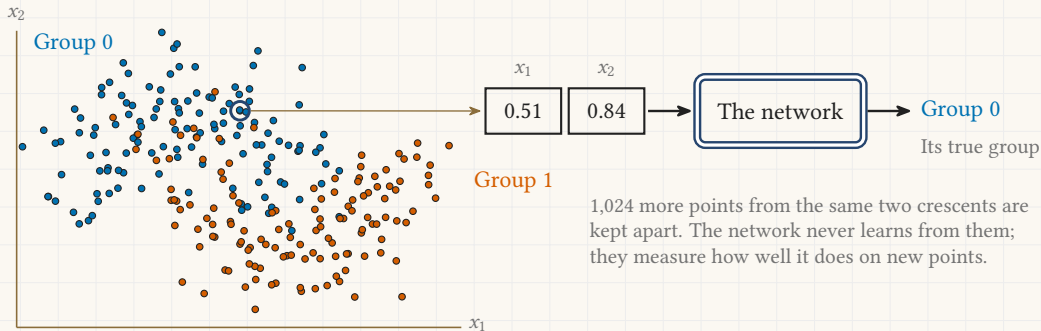
Two crescents, one network

Each point has two coordinates and belongs to one of two groups. The rest of the hour shows how to train a network that tells them apart, on a GPU of Santis.



Two crescents, one network

Each point has two coordinates and belongs to one of two groups. The rest of the hour shows how to train a network that tells them apart, on a GPU of Santis.



The roadmap

Two runs of one PyTorch file on a GPU of Santis, at width 16 and at width 128, in four steps.

1

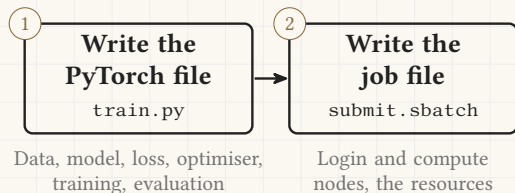
**Write the
PyTorch file**

`train.py`

Data, model, loss, optimiser,
training, evaluation

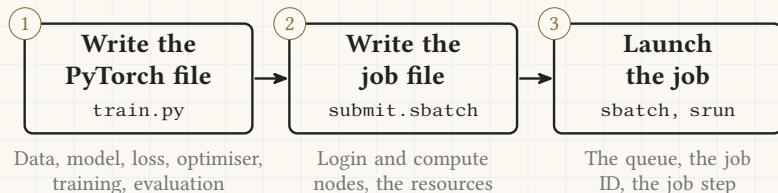
The roadmap

Two runs of one PyTorch file on a GPU of Santis, at width 16 and at width 128, in four steps.



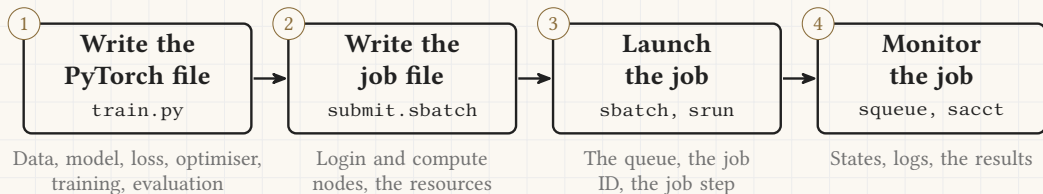
The roadmap

Two runs of one PyTorch file on a GPU of Santis, at width 16 and at width 128, in four steps.



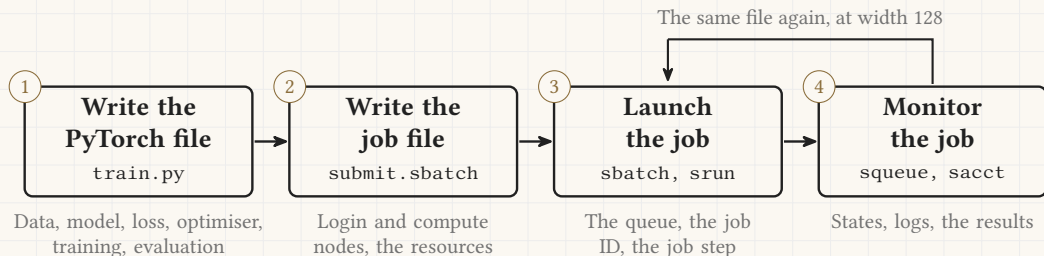
The roadmap

Two runs of one PyTorch file on a GPU of Santis, at width 16 and at width 128, in four steps.



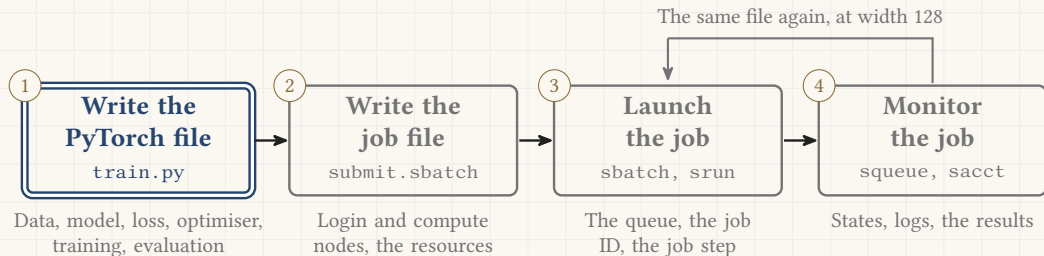
The roadmap

Two runs of one PyTorch file on a GPU of Santis, at width 16 and at width 128, in four steps.



The roadmap

Two runs of one PyTorch file on a GPU of Santis, at width 16 and at width 128, in four steps.



Four objects, and what each one does

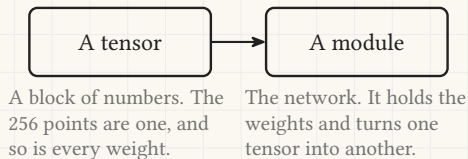
This morning's loop, as four PyTorch objects. `train.py` builds each of them, then runs the loop.

A tensor

A block of numbers. The 256 points are one, and so is every weight.

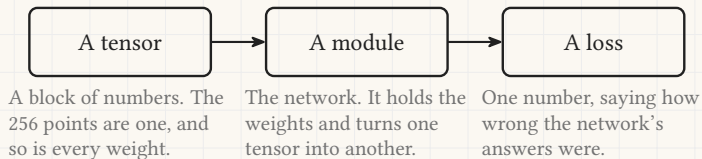
Four objects, and what each one does

This morning's loop, as four PyTorch objects. `train.py` builds each of them, then runs the loop.



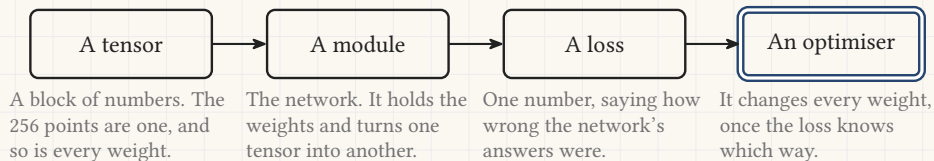
Four objects, and what each one does

This morning's loop, as four PyTorch objects. `train.py` builds each of them, then runs the loop.



Four objects, and what each one does

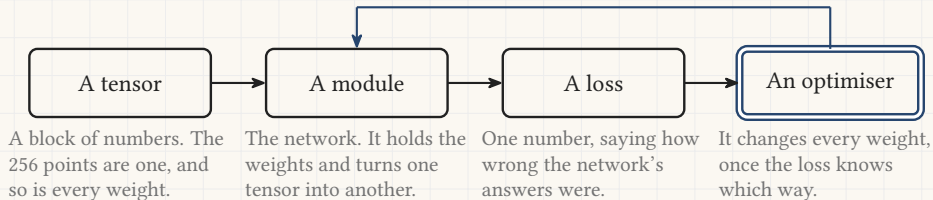
This morning's loop, as four PyTorch objects. `train.py` builds each of them, then runs the loop.



Four objects, and what each one does

This morning's loop, as four PyTorch objects. `train.py` builds each of them, then runs the loop.

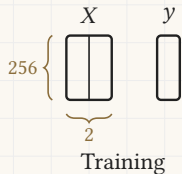
Round and round, 160 times. That is training.



The data, as two tensors

The first object. One call makes both sets, from the same two crescents.

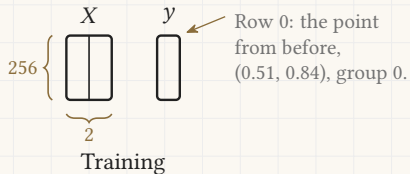
```
train_data, val_data = make_data()
```



The data, as two tensors

The first object. One call makes both sets, from the same two crescents.

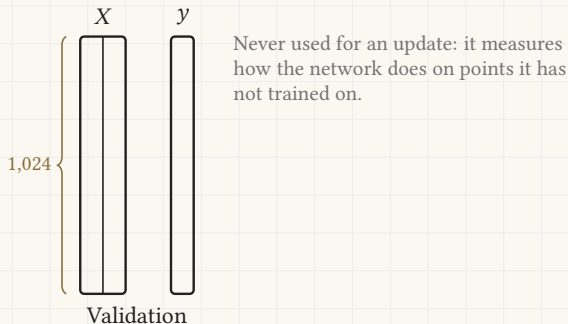
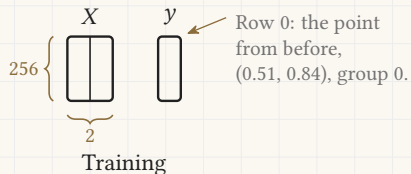
```
train_data, val_data = make_data()
```



The data, as two tensors

The first object. One call makes both sets, from the same two crescents.

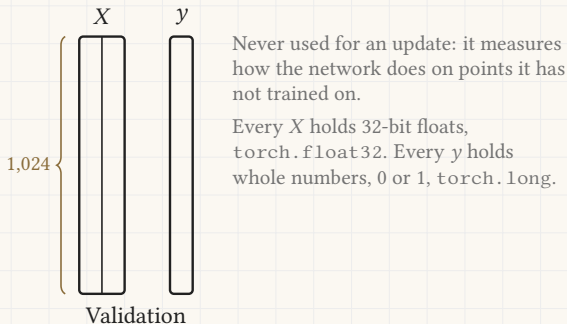
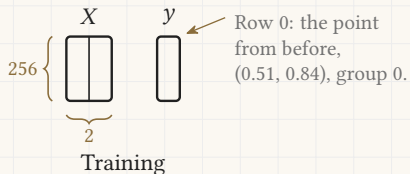
```
train_data, val_data = make_data()
```



The data, as two tensors

The first object. One call makes both sets, from the same two crescents.

```
train_data, val_data = make_data()
```



Minibatches of 64

This morning, one random house per round. Here, 64 random points per round.

```
train_loader = DataLoader(TensorDataset(*train_data), batch_size=args.batch_size,  
                           shuffle=True, generator=generator, num_workers=0)
```



X , 256 rows

Minibatches of 64

This morning, one random house per round. Here, 64 random points per round.

```
train_loader = DataLoader(TensorDataset(*train_data), batch_size=args.batch_size,  
                           shuffle=True, generator=generator, num_workers=0)
```

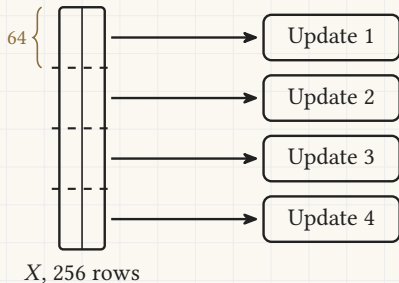


X , 256 rows

Minibatches of 64

This morning, one random house per round. Here, 64 random points per round.

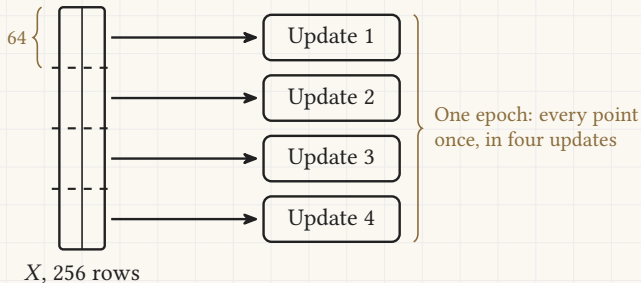
```
train_loader = DataLoader(TensorDataset(*train_data), batch_size=args.batch_size,  
                           shuffle=True, generator=generator, num_workers=0)
```



Minibatches of 64

This morning, one random house per round. Here, 64 random points per round.

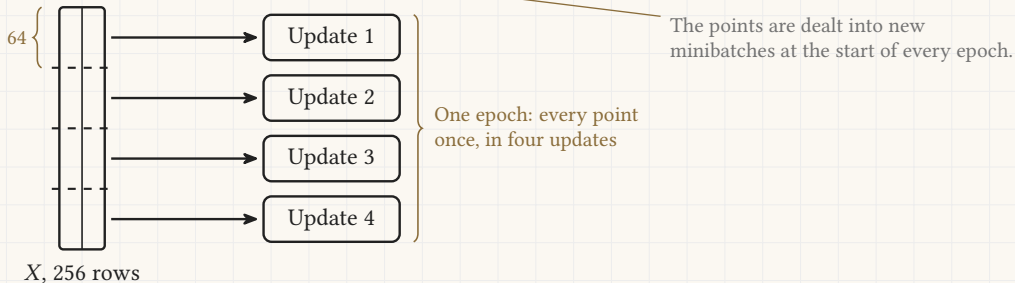
```
train_loader = DataLoader(TensorDataset(*train_data), batch_size=args.batch_size,  
                           shuffle=True, generator=generator, num_workers=0)
```



Minibatches of 64

This morning, one random house per round. Here, 64 random points per round.

```
train_loader = DataLoader(TensorDataset(*train_data), batch_size=args.batch_size,  
                           shuffle=True, generator=generator, num_workers=0)
```



The network, as blocks

The second object. One minibatch in, two scores per point out. Drawn at width 16: 16 numbers in each hidden layer.

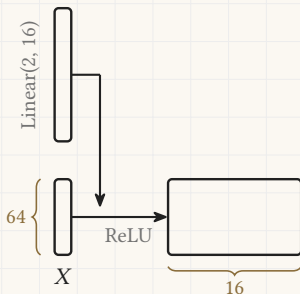
```
def make_model(width):  
    return nn.Sequential(nn.Linear(2, width), nn.ReLU(),  
                        nn.Linear(width, width), nn.ReLU(), nn.Linear(width, 2))
```



The network, as blocks

The second object. One minibatch in, two scores per point out. Drawn at width 16: 16 numbers in each hidden layer.

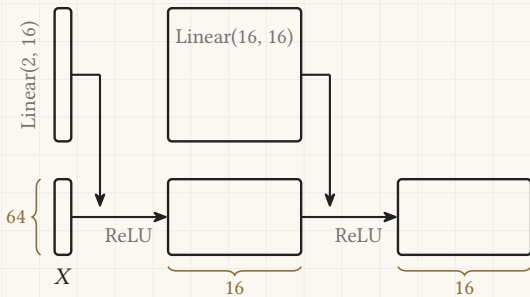
```
def make_model(width):  
    return nn.Sequential(nn.Linear(2, width), nn.ReLU(),  
                          nn.Linear(width, width), nn.ReLU(), nn.Linear(width, 2))
```



The network, as blocks

The second object. One minibatch in, two scores per point out. Drawn at width 16: 16 numbers in each hidden layer.

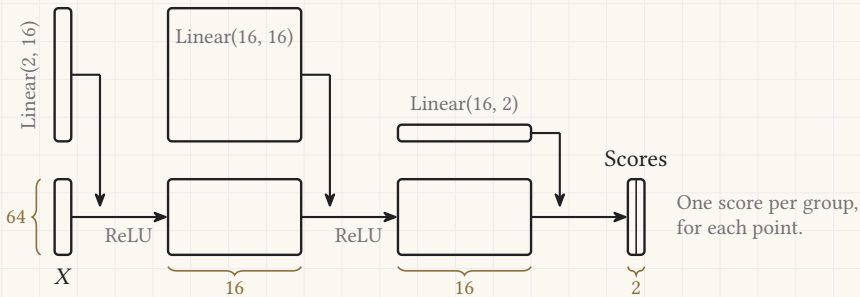
```
def make_model(width):  
    return nn.Sequential(nn.Linear(2, width), nn.ReLU(),  
                          nn.Linear(width, width), nn.ReLU(), nn.Linear(width, 2))
```



The network, as blocks

The second object. One minibatch in, two scores per point out. Drawn at width 16: 16 numbers in each hidden layer.

```
def make_model(width):  
    return nn.Sequential(nn.Linear(2, width), nn.ReLU(),  
                          nn.Linear(width, width), nn.ReLU(), nn.Linear(width, 2))
```



The numbers this network holds

Every number the optimiser will change, both widths drawn to one scale. The two runs differ in nothing else.



Width 16

The numbers this network holds

Every number the optimiser will change, both widths drawn to one scale. The two runs differ in nothing else.

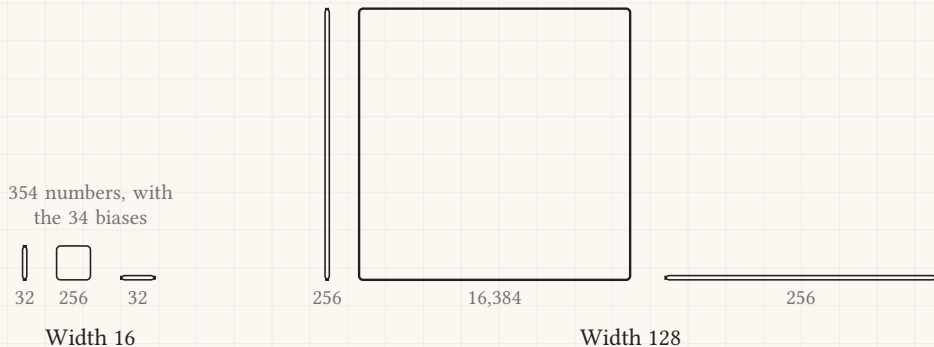
354 numbers, with
the 34 biases



Width 16

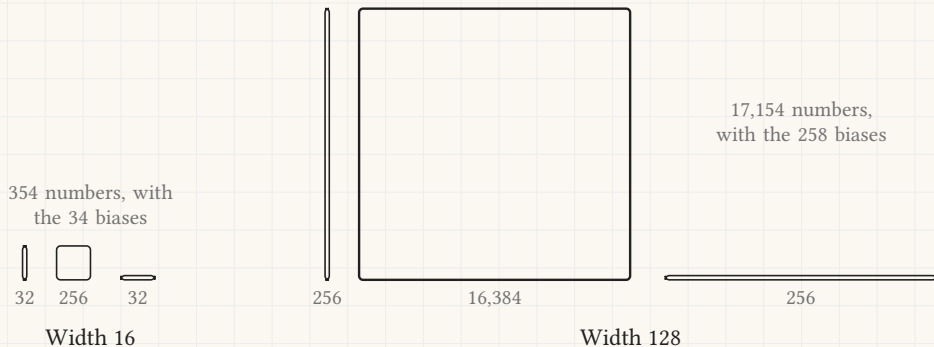
The numbers this network holds

Every number the optimiser will change, both widths drawn to one scale. The two runs differ in nothing else.



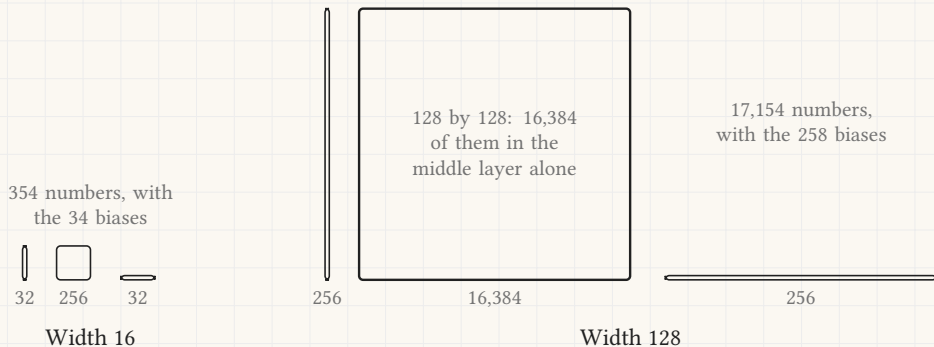
The numbers this network holds

Every number the optimiser will change, both widths drawn to one scale. The two runs differ in nothing else.



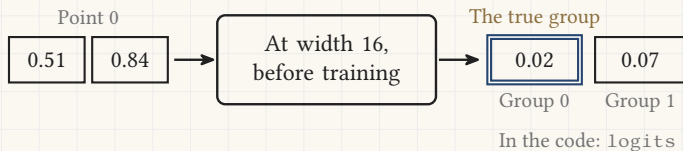
The numbers this network holds

Every number the optimiser will change, both widths drawn to one scale. The two runs differ in nothing else.



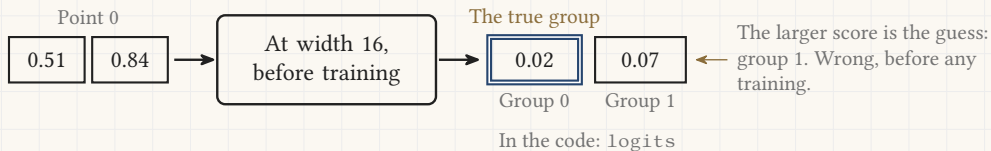
Two scores out, one verdict

The third object. This morning's verdict was a squared miss, `nn.MSELoss()`. Here the network gives one score per group.



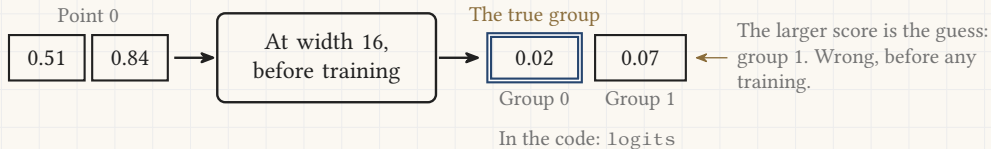
Two scores out, one verdict

The third object. This morning's verdict was a squared miss, `nn.MSELoss()`. Here the network gives one score per group.



Two scores out, one verdict

The third object. This morning's verdict was a squared miss, `nn.MSELoss()`. Here the network gives one score per group.



```
loss_fn = nn.CrossEntropyLoss()
```

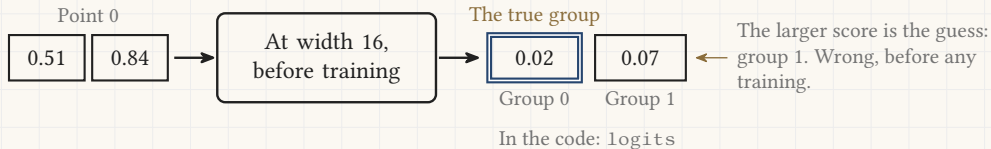
```
loss = loss_fn(logits, y)
```

0.72

The loss for point 0

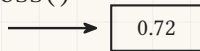
Two scores out, one verdict

The third object. This morning's verdict was a squared miss, `nn.MSELoss()`. Here the network gives one score per group.



```
loss_fn = nn.CrossEntropyLoss()
```

```
loss = loss_fn(logits, y)
```



The loss for point 0

Large when the true group got the smaller score. Training moves the weights to make it smaller.

The loss and the optimiser

The fourth object, and the two lines of `train.py` that create the last two.

```
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr,  
                               weight_decay=args.weight_decay)  
loss_fn = nn.CrossEntropyLoss()
```

The loss and the optimiser

The fourth object, and the two lines of `train.py` that create the last two.

The network's 354 numbers,
17,154 at width 128



```
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr,  
                               weight_decay=args.weight_decay)  
loss_fn = nn.CrossEntropyLoss()
```

The loss and the optimiser

The fourth object, and the two lines of `train.py` that create the last two.

The network's 354 numbers,
17,154 at width 128

This morning's α , now
scaling every step: 0.01
by default

```
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr,  
                             weight_decay=args.weight_decay)  
loss_fn = nn.CrossEntropyLoss()
```

The loss and the optimiser

The fourth object, and the two lines of `train.py` that create the last two.

Adam: each number gets a
step size of its own, tuned
from its own gradients

The network's 354 numbers,
17,154 at width 128

This morning's α , now
scaling every step: 0.01
by default



```
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr,  
                             weight_decay=args.weight_decay)  
loss_fn = nn.CrossEntropyLoss()
```

One training step

The four objects in one loop, one minibatch at a time: the function `training_step`.

Minibatch

`x, y = batch`

One training step

The four objects in one loop, one minibatch at a time: the function `training_step`.

Minibatch

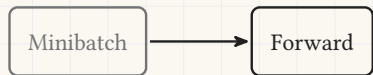
Backward
adds to old
gradients,
so clear
them first.

```
x, y = batch
```

```
optimizer.zero_grad()
```

One training step

The four objects in one loop, one minibatch at a time: the function `training_step`.



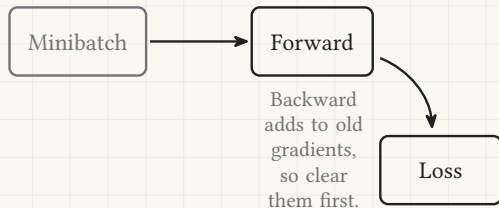
Backward
adds to old
gradients,
so clear
them first.

```
x, y = batch
```

```
optimizer.zero_grad()  
logits = model(x)
```

One training step

The four objects in one loop, one minibatch at a time: the function `training_step`.



```
x, y = batch
```

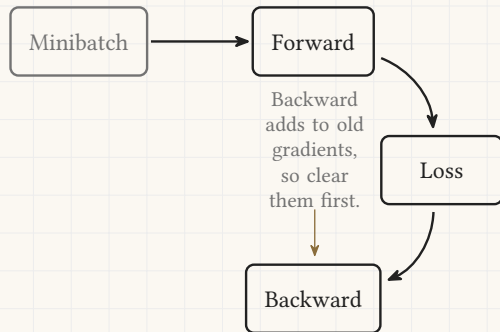
```
optimizer.zero_grad()
```

```
logits = model(x)
```

```
loss = loss_fn(logits, y)
```

One training step

The four objects in one loop, one minibatch at a time: the function `training_step`.

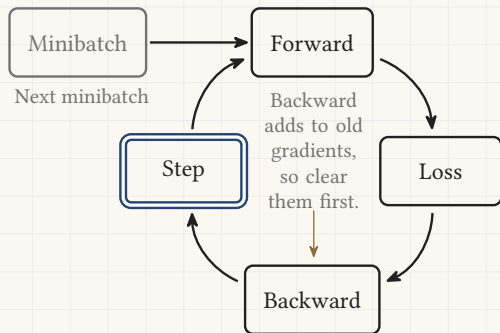


```
x, y = batch
```

```
optimizer.zero_grad()
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()
```


One training step

The four objects in one loop, one minibatch at a time: the function `training_step`.

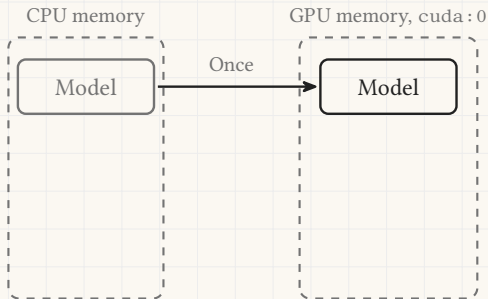


```
x, y = batch
```

```
optimizer.zero_grad()  
logits = model(x)  
loss = loss_fn(logits, y)  
loss.backward()  
optimizer.step()
```

Model and minibatch on the same device

The same step, with the line that moves each minibatch to the GPU, where the model already is.



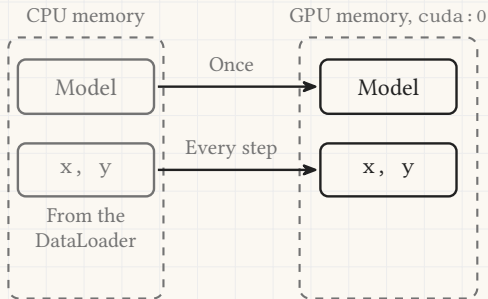
```
x, y = batch
x, y = x.to(device), y.to(device)
optimizer.zero_grad()
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()
optimizer.step()
```

`model = make_model(args.width).to(device)`

In `train.py`, once, before the optimiser is built

Model and minibatch on the same device

The same step, with the line that moves each minibatch to the GPU, where the model already is.



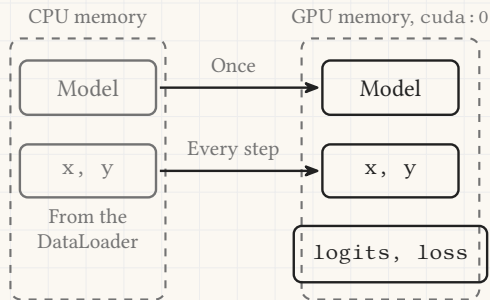
```
x, y = batch
x, y = x.to(device), y.to(device)
optimizer.zero_grad()
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()
optimizer.step()
```

```
model = make_model(args.width).to(device)
```

In `train.py`, once, before the optimiser is built

Model and minibatch on the same device

The same step, with the line that moves each minibatch to the GPU, where the model already is.



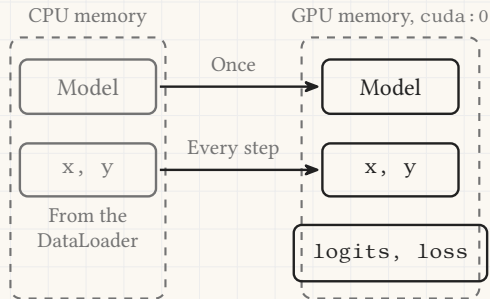
```
x, y = batch
x, y = x.to(device), y.to(device)
optimizer.zero_grad()
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()
optimizer.step()
```

```
model = make_model(args.width).to(device)
```

In `train.py`, once, before the optimiser is built

Model and minibatch on the same device

The same step, with the line that moves each minibatch to the GPU, where the model already is.



`train.py` picks `cuda:0` only inside a job, with exactly one GPU visible.

```
x, y = batch
x, y = x.to(device), y.to(device)
optimizer.zero_grad()
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()
optimizer.step()
```

```
model = make_model(args.width).to(device)
```

In `train.py`, once, before the optimiser is built

Forty epochs of four steps

The loop around the step. After every epoch, both sets are measured with the same weights.

```
for epoch in range(1, args.epochs + 1):  
    model.train()  
    for batch in train_loader:  
        loss, evidence = training_step(model, batch, optimizer, loss_fn, device)
```

Epoch 1



Epoch 2



...

Epoch 40



Forty epochs of four steps

The loop around the step. After every epoch, both sets are measured with the same weights.

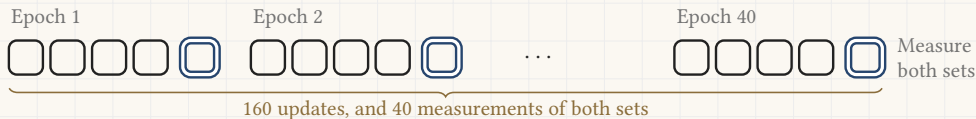
```
for epoch in range(1, args.epochs + 1):  
    model.train()  
    for batch in train_loader:  
        loss, evidence = training_step(model, batch, optimizer, loss_fn, device)  
    tr_loss, tr_acc = evaluate(model, train_eval, device)  
    va_loss, va_acc = evaluate(model, val_loader, device)
```



Forty epochs of four steps

The loop around the step. After every epoch, both sets are measured with the same weights.

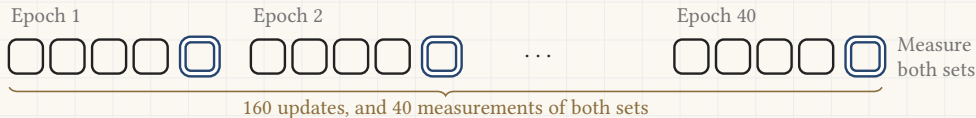
```
for epoch in range(1, args.epochs + 1):  
    model.train()  
    for batch in train_loader:  
        loss, evidence = training_step(model, batch, optimizer, loss_fn, device)  
    tr_loss, tr_acc = evaluate(model, train_eval, device)  
    va_loss, va_acc = evaluate(model, val_loader, device)
```



Forty epochs of four steps

The loop around the step. After every epoch, both sets are measured with the same weights.

```
for epoch in range(1, args.epochs + 1):  
    model.train() ← Back to training behaviour at the start of every epoch  
    for batch in train_loader:  
        loss, evidence = training_step(model, batch, optimizer, loss_fn, device)  
    tr_loss, tr_acc = evaluate(model, train_eval, device)  
    va_loss, va_acc = evaluate(model, val_loader, device)
```



Measuring without learning

`evaluate()` runs the network over a whole set and returns its loss and accuracy, without changing a weight.

```
@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    total_loss, correct, count = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        total_loss += nn.functional.cross_entropy(logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item()
        count += len(y)
    return total_loss / count, correct / count
```

Measuring without learning

`evaluate()` runs the network over a whole set and returns its loss and accuracy, without changing a weight.

`@torch.no_grad()`



No computation graph is built, which saves memory and time

```
def evaluate(model, loader, device):
    model.eval()
    total_loss, correct, count = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        total_loss += nn.functional.cross_entropy(logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item()
        count += len(y)
    return total_loss / count, correct / count
```

Measuring without learning

`evaluate()` runs the network over a whole set and returns its loss and accuracy, without changing a weight.

```
@torch.no_grad()  ← No computation graph is built, which saves
def evaluate(model, loader, device):                      memory and time
    model.eval()    ← Evaluation behaviour for every layer; none
                    of this network's layers changes with it
    total_loss, correct, count = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        total_loss += nn.functional.cross_entropy(logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item()
        count += len(y)
    return total_loss / count, correct / count
```

Measuring without learning

`evaluate()` runs the network over a whole set and returns its loss and accuracy, without changing a weight.

```
@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    total_loss, correct, count = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        total_loss += nn.functional.cross_entropy(logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item()
        count += len(y)
    return total_loss / count, correct / count
```

← No computation graph is built, which saves memory and time

← Evaluation behaviour for every layer; none of this network's layers changes with it

← Correct when the larger score belongs to the true group

Measuring without learning

`evaluate()` runs the network over a whole set and returns its loss and accuracy, without changing a weight.

```
@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    total_loss, correct, count = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        total_loss += nn.functional.cross_entropy(logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item()
        count += len(y)
    return total_loss / count, correct / count
```

← No computation graph is built, which saves memory and time

← Evaluation behaviour for every layer; none of this network's layers changes with it

← Correct when the larger score belongs to the true group

No optimizer.step() anywhere: evaluation only reads

Layer behaviour and gradient tracking are separate

The two switches of the last slide do different jobs, and evaluation code uses both.

```
model.eval()
```

*Dropout turns off; BatchNorm
switches to its running statistics*

Layer behaviour and gradient tracking are separate

The two switches of the last slide do different jobs, and evaluation code uses both.

```
model.eval()
```

*Dropout turns off; BatchNorm
switches to its running statistics*

Says nothing about gradients

Layer behaviour and gradient tracking are separate

The two switches of the last slide do different jobs, and evaluation code uses both.

`model.eval()`

Dropout turns off; BatchNorm switches to its running statistics

Says nothing about gradients

`torch.no_grad()`

Stops autograd building the computation graph

Layer behaviour and gradient tracking are separate

The two switches of the last slide do different jobs, and evaluation code uses both.

`model.eval()`

Dropout turns off; BatchNorm switches to its running statistics

Says nothing about gradients

`torch.no_grad()`

Stops autograd building the computation graph

Saves the memory and compute gradients cost

The file takes its settings as flags

The file is complete. What the two runs change arrives on the command line.

```
p.add_argument("--width", type=int, default=16)
p.add_argument("--epochs", type=int, default=40)
p.add_argument("--batch-size", type=int, default=64)
p.add_argument("--lr", type=float, default=0.01)
p.add_argument("--weight-decay", type=float, default=0.0)
p.add_argument("--seed", type=int, default=42)
p.add_argument("--output", type=Path, required=True)
```

The file takes its settings as flags

The file is complete. What the two runs change arrives on the command line.

```
p.add_argument("--width", type=int, default=16)
p.add_argument("--epochs", type=int, default=40)
p.add_argument("--batch-size", type=int, default=64)
p.add_argument("--lr", type=float, default=0.01)
p.add_argument("--weight-decay", type=float, default=0.0)
p.add_argument("--seed", type=int, default=42)
p.add_argument("--output", type=Path, required=True)
```

← The one setting the two runs
change: 16, then 128

The file takes its settings as flags

The file is complete. What the two runs change arrives on the command line.

```
p.add_argument("--width", type=int, default=16)
p.add_argument("--epochs", type=int, default=40)
p.add_argument("--batch-size", type=int, default=64)
p.add_argument("--lr", type=float, default=0.01)
p.add_argument("--weight-decay", type=float, default=0.0)
p.add_argument("--seed", type=int, default=42)
p.add_argument("--output", type=Path, required=True)
```

← The one setting the two runs
change: 16, then 128

} Left at their defaults, the same in
both runs

The file takes its settings as flags

The file is complete. What the two runs change arrives on the command line.

```
p.add_argument("--width", type=int, default=16)
p.add_argument("--epochs", type=int, default=40)
p.add_argument("--batch-size", type=int, default=64)
p.add_argument("--lr", type=float, default=0.01)
p.add_argument("--weight-decay", type=float, default=0.0)
p.add_argument("--seed", type=int, default=42)
p.add_argument("--output", type=Path, required=True)
```

← The one setting the two runs
change: 16, then 128

} Left at their defaults, the same in
both runs

← A new folder for every run; an
existing one is refused

In it:

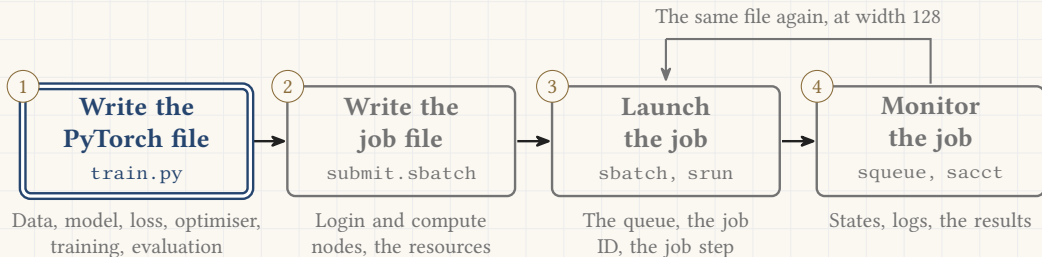
config.json

metrics.csv

summary.json

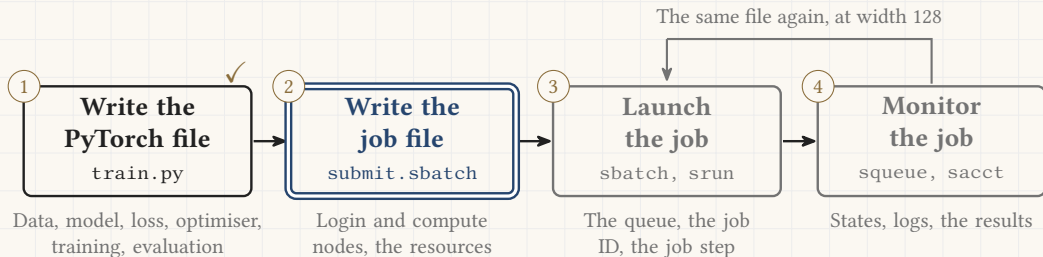
Step 2 of 4

Write the job file



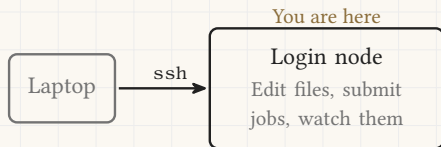
Step 2 of 4

Write the job file



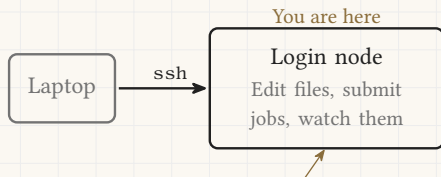
Login node and compute nodes

Where you log in is not where you train, and `train.py` knows it.



Login node and compute nodes

Where you log in is not where you train, and `train.py` knows it.

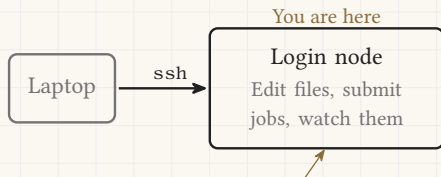


Started outside a job, `train.py` refuses to train:

No Slurm allocation. Submit `submit.sbatch`; do not train on a login node.

Login node and compute nodes

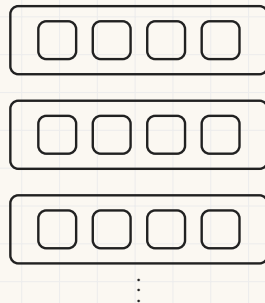
Where you log in is not where you train, and `train.py` knows it.



Started outside a job, `train.py` refuses to train:

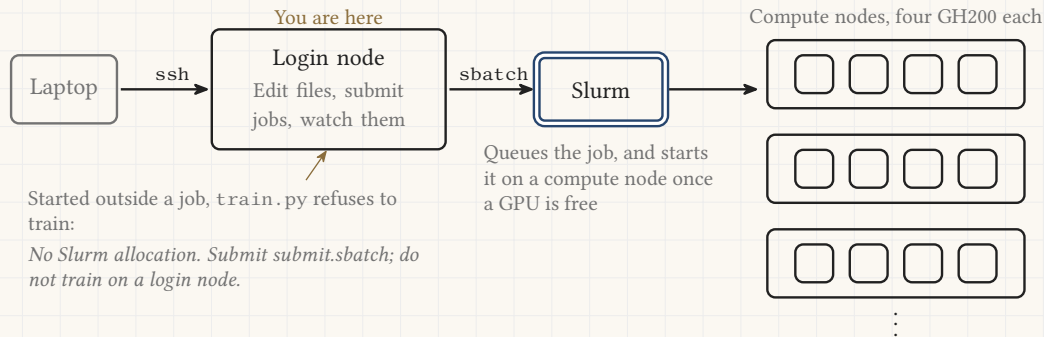
No Slurm allocation. Submit `submit.sbatch`; do not train on a login node.

Compute nodes, four GH200 each



Login node and compute nodes

Where you log in is not where you train, and `train.py` knows it.



The job file asks for resources

The top of `submit.sbatch`: requests that Slurm reads before anything runs.

```
submit.sbatch
#!/bin/bash
#SBATCH --job-name=cx-pytorch
#SBATCH --account=ai-tutorial-course2026-cscs
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=<NTASKS>
#SBATCH --gpus-per-task=<GPUS_PER_TASK>
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SBATCH --uenv=pytorch/v2.9.1:v2
#SBATCH --view=default
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

The job file asks for resources

The top of `submit.sbatch`: requests that Slurm reads before anything runs.

```
submit.sbatch
#!/bin/bash
#SBATCH --job-name=cx-pytorch
#SBATCH --account=ai-tutorial-course2026-cscs
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=<NTASKS>
#SBATCH --gpus-per-task=<GPUS_PER_TASK>
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SBATCH --uenv=pytorch/v2.9.1:v2
#SBATCH --view=default
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

What the file asks for

Paid by the course account, queued in the normal partition

The job file asks for resources

The top of `submit.sbatch`: requests that Slurm reads before anything runs.

```
submit.sbatch
```

```
#!/bin/bash
#SBATCH --job-name=cx-pytorch
#SBATCH --account=ai-tutorial-course2026-cscs
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=<NTASKS>
#SBATCH --gpus-per-task=<GPUS_PER_TASK>
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SBATCH --uenv=pytorch/v2.9.1:v2
#SBATCH --view=default
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

What the file asks for

Paid by the course account, queued in the normal partition

One node; one task, the one Python process

The job file asks for resources

The top of `submit.sbatch`: requests that Slurm reads before anything runs.

```
submit.sbatch
```

```
#!/bin/bash
#SBATCH --job-name=cx-pytorch
#SBATCH --account=ai-tutorial-course2026-cscs
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=<NTASKS>
#SBATCH --gpus-per-task=<GPUS_PER_TASK>
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SBATCH --uenv=pytorch/v2.9.1:v2
#SBATCH --view=default
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

What the file asks for

Paid by the course account, queued in the normal partition

One node; one task, the one Python process

One GH200

One core

Without a GPU request: one core, no GPU, and `train.py` stops

The job file asks for resources

The top of `submit.sbatch`: requests that Slurm reads before anything runs.

`submit.sbatch`

```
#!/bin/bash
#SBATCH --job-name=cx-pytorch
#SBATCH --account=ai-tutorial-course2026-cscs
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=<NTASKS>
#SBATCH --gpus-per-task=<GPUS_PER_TASK>
#SBATCH --cpus-per-task=1
#SSBATCH --time=00:05:00
#SBATCH --uenv=pytorch/v2.9.1:v2
#SBATCH --view=default
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

What the file asks for

Paid by the course account, queued in the normal partition

One node; one task, the one Python process

One GH200

One core

Without a GPU request: one core, no GPU, and `train.py` stops

At most five minutes, then Slurm stops the job

The job file asks for resources

The top of `submit.sbatch`: requests that Slurm reads before anything runs.

`submit.sbatch`

```
#!/bin/bash
#SBATCH --job-name=cx-pytorch
#SBATCH --account=ai-tutorial-course2026-cscs
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=<NTASKS>
#SBATCH --gpus-per-task=<GPUS_PER_TASK>
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SUBATCH --uenv=pytorch/v2.9.1:v2
#SUBATCH --view=default
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

What the file asks for

Paid by the course account, queued in the normal partition

One node; one task, the one Python process

One GH200

One core

Without a GPU request: one core, no GPU, and `train.py` stops

At most five minutes, then Slurm stops the job
PyTorch from a CSCS software image, a uenv

The job file asks for resources

The top of `submit.sbatch`: requests that Slurm reads before anything runs.

`submit.sbatch`

```
#!/bin/bash
#SBATCH --job-name=cx-pytorch
#SBATCH --account=ai-tutorial-course2026-cscs
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=<NTASKS>
#SBATCH --gpus-per-task=<GPUS_PER_TASK>
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SBATCH --uenv=pytorch/v2.9.1:v2
#SBATCH --view=default
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

What the file asks for

Paid by the course account, queued in the normal partition

One node; one task, the one Python process

One GH200

One core

Without a GPU request: one core, no GPU, and `train.py` stops

At most five minutes, then Slurm stops the job
PyTorch from a CSCS software image, a uenv
Log names: %x the job name, %j its number

The job file then runs the script

Below the requests, ordinary shell lines. Slurm runs them on the compute node, once the job starts.

```
submit.sbatch, below the requests
cd "$SLURM_SUBMIT_DIR"
width="${1:-16}"
weight_decay="${2:-0.0}"
output="runs/${SLURM_JOB_ID}"
srun python -u train.py \
    --width "$width" --weight-decay "$weight_decay" --output "$output"
```

The job file then runs the script

Below the requests, ordinary shell lines. Slurm runs them on the compute node, once the job starts.

```
ooo
```

```
$ job1=$(sbatch --parsable submit.sbatch 16)
```

submit.sbatch, below the requests

```
cd "$SLURM_SUBMIT_DIR"
```

```
width="${1:-16}" ← The first argument after the file name: 16, and later 128
```

```
weight_decay="${2:-0.0}"
```

```
output="runs/${SLURM_JOB_ID}"
```

```
srun python -u train.py \
```

```
    --width "$width" --weight-decay "$weight_decay" --output "$output"
```

The job file then runs the script

Below the requests, ordinary shell lines. Slurm runs them on the compute node, once the job starts.

```
ooo
```

```
$ job1=$(sbatch --parsable submit.sbatch 16)
```

submit.sbatch, below the requests

```
cd "$SLURM_SUBMIT_DIR"
```

```
width="${1:-16}" ← The first argument after the file name: 16, and later 128
```

```
weight_decay="${2:-0.0}"
```

```
output="runs/${SLURM_JOB_ID}" ← Set by Slurm: one folder per job
```

```
srun python -u train.py \  
    --width "$width" --weight-decay "$weight_decay" --output "$output"
```

The job file then runs the script

Below the requests, ordinary shell lines. Slurm runs them on the compute node, once the job starts.

```
ooo
```

```
$ job1=$(sbatch --parsable submit.sbatch 16)
```

submit.sbatch, below the requests

```
cd "$SLURM_SUBMIT_DIR"
```

```
width="${1:-16}" ← The first argument after the file name: 16, and later 128
```

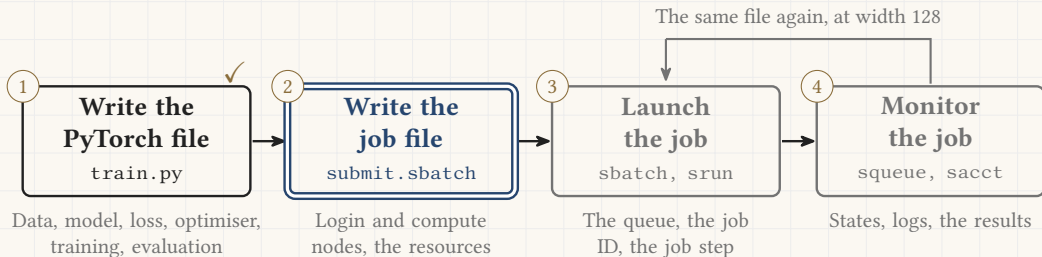
```
weight_decay="${2:-0.0}"
```

```
output="runs/${SLURM_JOB_ID}" ← Set by Slurm: one folder per job
```

```
srun python -u train.py \ ← Starts train.py inside the allocation, as a job step  
--width "$width" --weight-decay "$weight_decay" --output "$output"
```

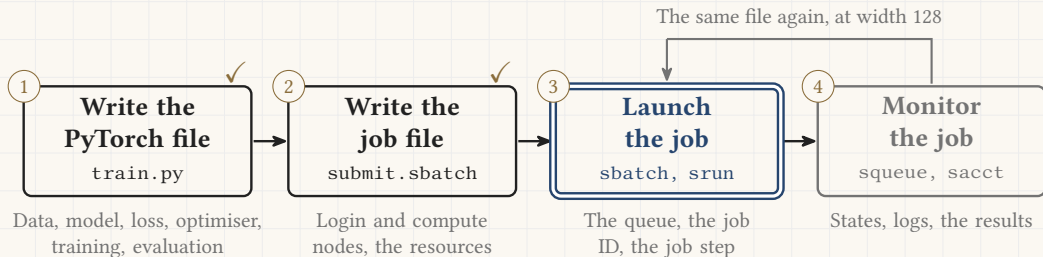
Step 3 of 4

Launch the job



Step 3 of 4

Launch the job



sbatch puts the job in the queue

Typed on the login node. It returns at once, with the job's number.

```
ooo
```

```
$ mkdir -p logs
```



First: Slurm opens the log files
before the script runs, so without
logs/ the job leaves no log at all

sbatch puts the job in the queue

Typed on the login node. It returns at once, with the job's number.

```
ooo
```

```
$ mkdir -p logs
```

```
$ job1=$(sbatch --parsable submit.sbatch 16)
```

First: Slurm opens the log files
before the script runs, so without
logs/ the job leaves no log at all

sbatch puts the job in the queue

Typed on the login node. It returns at once, with the job's number.

ooo

```
$ mkdir -p logs  
$ job1=$(sbatch --parsable submit.sbatch 16)  
$ job1=${job1%%;*}    Keeps the number alone  
$ printf 'Baseline job ID: %s\n' "$job1"  
Baseline job ID: <JOB_ID>
```

First: Slurm opens the log files
before the script runs, so without
logs/ the job leaves no log at all

sbatch puts the job in the queue

Typed on the login node. It returns at once, with the job's number.

```
ooo
$ mkdir -p logs
$ job1=$(sbatch --parsable submit.sbatch 16)
$ job1=${job1%%;*}    Keeps the number alone
$ printf 'Baseline job ID: %s\n' "$job1"
Baseline job ID: <JOB_ID>
```

First: Slurm opens the log files
before the script runs, so without
logs/ the job leaves no log at all

The queue



Your job



PENDING: waiting for a free GPU



Towards a free GPU

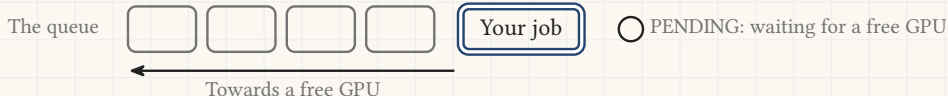
sbatch puts the job in the queue

Typed on the login node. It returns at once, with the job's number.

```
ooo
$ mkdir -p logs
$ job1=$(sbatch --parsable submit.sbatch 16)
$ job1=${job1%%;*}    Keeps the number alone
$ printf 'Baseline job ID: %s\n' "$job1"
Baseline job ID: <JOB_ID>
```

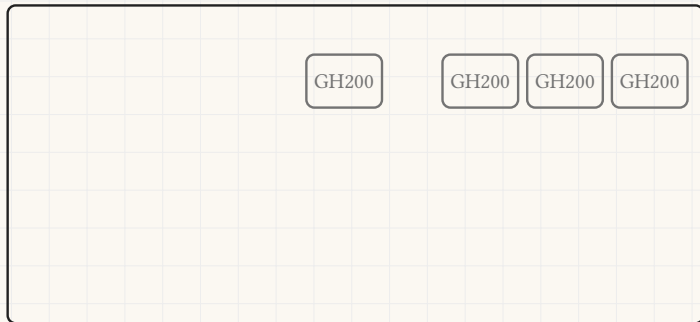
First: Slurm opens the log files before the script runs, so without logs/ the job leaves no log at all

A job ID confirms submission, not completion.



srun starts the training

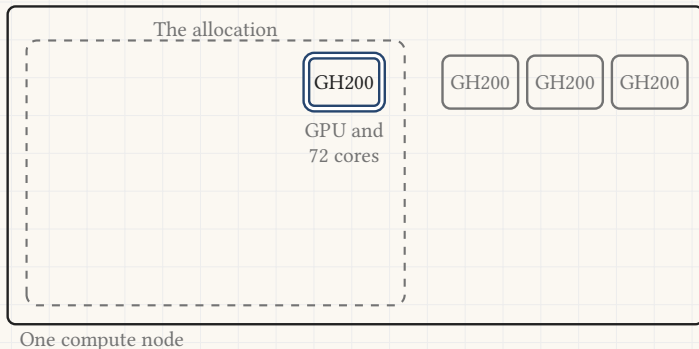
When a GPU is free, Slurm runs `submit.sbatch` on a compute node, and its `srun` line starts `train.py`.



One compute node

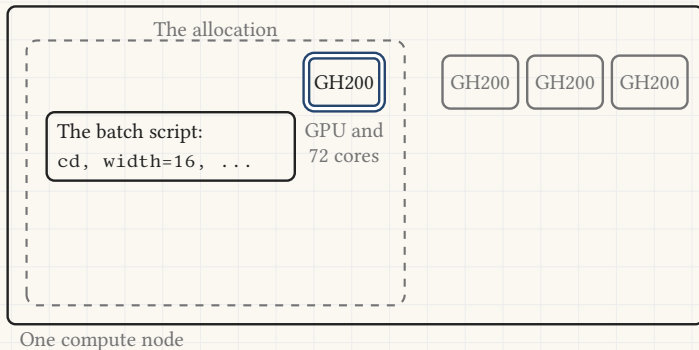
srun starts the training

When a GPU is free, Slurm runs `submit.sbatch` on a compute node, and its `srun` line starts `train.py`.



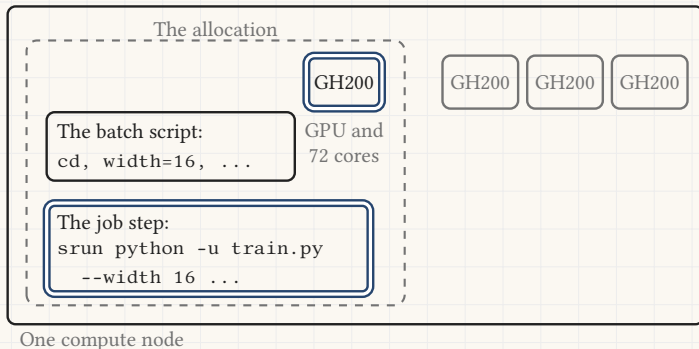
srun starts the training

When a GPU is free, Slurm runs `submit.sbatch` on a compute node, and its `srun` line starts `train.py`.



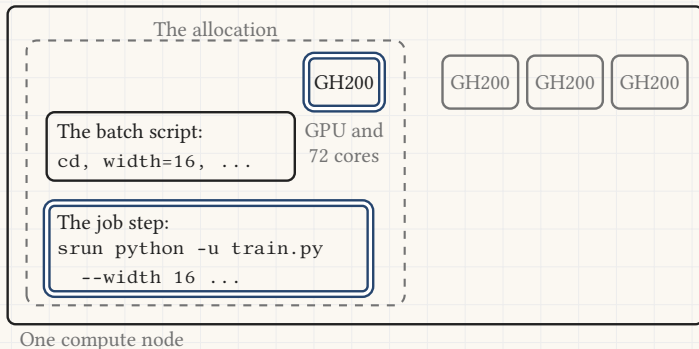
srun starts the training

When a GPU is free, Slurm runs `submit.sbatch` on a compute node, and its `srun` line starts `train.py`.



srun starts the training

When a GPU is free, Slurm runs `submit.sbatch` on a compute node, and its `srun` line starts `train.py`.



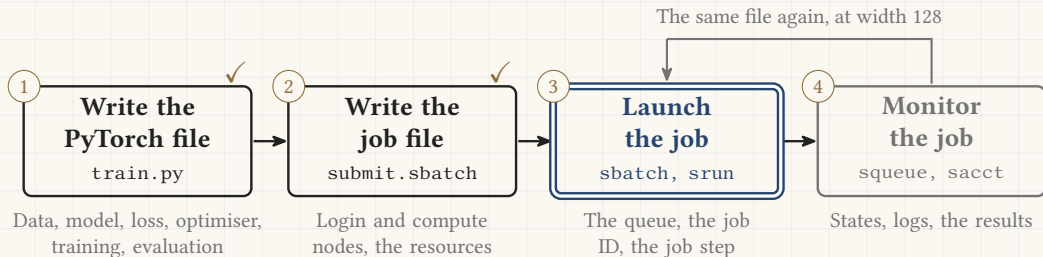
Inside the job, the checks of `train.py` pass:

- ✓ `SLURM_JOB_ID` is set: this is an allocation
- ✓ `CUDA` works and exactly one GPU is visible: the device is `cuda:0`

So the 160 updates run on the GH200.

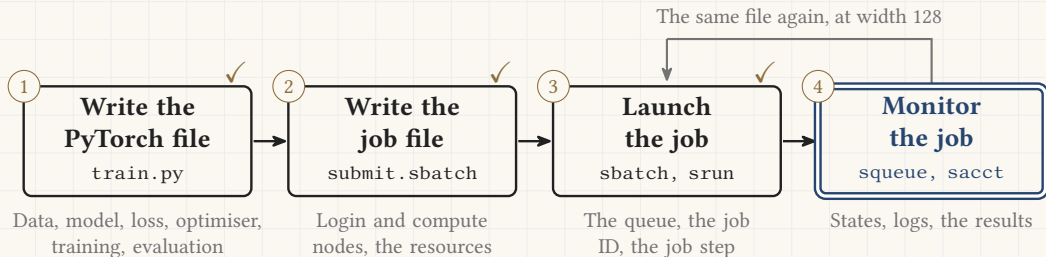
Step 4 of 4

Monitor the job



Step 4 of 4

Monitor the job



squeue shows where the job stands

While the job is in the queue, squeue shows its state; scontrol explains a wait.

```
ooo
```

```
$ squeue -j "$job1" -o '%.18i %.9T %.10M %.6D %R'
```

squeue shows where the job stands

While the job is in the queue, squeue shows its state; scontrol explains a wait.

```
ooo
```

```
$ squeue -j "$job1" -o '%.18i %.9T %.10M %.6D %R'
```

PENDING



Waiting, with a reason such
as Resources or Priority

squeue shows where the job stands

While the job is in the queue, `squeue` shows its state; `scontrol` explains a wait.

```
ooo
```

```
$ squeue -j "$job1" -o '%.18i %.9T %.10M %.6D %R'
```

PENDING



Waiting, with a reason such
as Resources or Priority

RUNNING



On a compute
node; the log grows

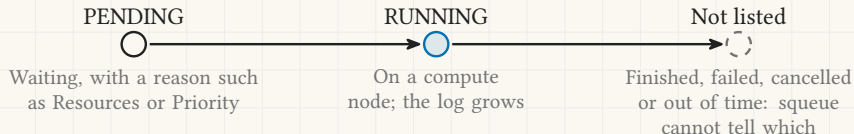


squeue shows where the job stands

While the job is in the queue, squeue shows its state; scontrol explains a wait.

```
ooo
```

```
$ squeue -j "$job1" -o '%.18i %.9T %.10M %.6D %R'
```



queue shows where the job stands

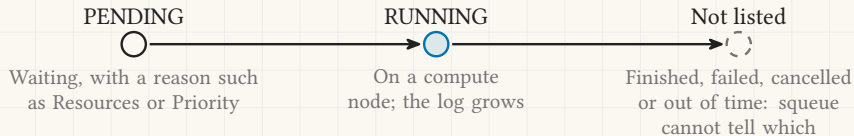
While the job is in the queue, `squeue` shows its state; `scontrol` explains a wait.

```
ooo
```

```
$ squeue -j "$job1" -o '%.18i %.9T %.10M %.6D %R'
```

```
$ scontrol show job "$job1"
```

Every detail of one job, the reason for a wait among them. Pending past two minutes: ask, do not resubmit.



Gone from the queue is not success

The logs and sacct say how the job ended.

ooo

```
$ tail -n 20 "logs/cx-pytorch-${job1}.out"
```

What `train.py` printed, epoch by epoch

Gone from the queue is not success

The logs and sacct say how the job ended.

ooo

```
$ tail -n 20 "logs/cx-pytorch-${job1}.out"
```

What `train.py` printed, epoch by epoch

```
$ tail -n 20 "logs/cx-pytorch-${job1}.err"
```

Errors and warnings

Gone from the queue is not success

The logs and sacct say how the job ended.

ooo

```
$ tail -n 20 "logs/cx-pytorch-${job1}.out"      What train.py printed, epoch by epoch
$ tail -n 20 "logs/cx-pytorch-${job1}.err"      Errors and warnings
$ sacct -j "$job1" --format=JobID,State,ExitCode,Elapsed,AllocTRES
```

Gone from the queue is not success

The logs and sacct say how the job ended.

ooo

```
$ tail -n 20 "logs/cx-pytorch-${job1}.out"      What train.py printed, epoch by epoch
$ tail -n 20 "logs/cx-pytorch-${job1}.err"      Errors and warnings
$ sacct -j "$job1" --format=JobID,State,ExitCode,Elapsed,AllocTRES
```

- COMPLETED, exit code 0:0
And summary.json exists

Gone from the queue is not success

The logs and sacct say how the job ended.

ooo

```
$ tail -n 20 "logs/cx-pytorch-${job1}.out"      What train.py printed, epoch by epoch
$ tail -n 20 "logs/cx-pytorch-${job1}.err"      Errors and warnings
$ sacct -j "$job1" --format=JobID,State,ExitCode,Elapsed,AllocTRES
```



COMPLETED, exit code 0:0

And summary.json exists



FAILED, TIMEOUT or CANCELLED

Read the .err log first

Gone from the queue is not success

The logs and sacct say how the job ended.

ooo

```
$ tail -n 20 "logs/cx-pytorch-${job1}.out"      What train.py printed, epoch by epoch
$ tail -n 20 "logs/cx-pytorch-${job1}.err"      Errors and warnings
$ sacct -j "$job1" --format=JobID,State,ExitCode,Elapsed,AllocTRES
$ scancel <JOB_ID>    Cancels a job by its number, waiting or running
```



COMPLETED, exit code 0:0

And summary.json exists



FAILED, TIMEOUT or CANCELLED

Read the .err log first

What the job leaves behind

Every file carries the job's number, so two runs never mix.

`logs/cx-pytorch-<JOB_ID>.out` The epochs, and the devices of the first step

What the job leaves behind

Every file carries the job's number, so two runs never mix.

<code>logs/cx-pytorch-<JOB_ID>.out</code>	The epochs, and the devices of the first step
<code>logs/cx-pytorch-<JOB_ID>.err</code>	Errors and warnings

What the job leaves behind

Every file carries the job's number, so two runs never mix.

<code>logs/cx-pytorch-<JOB_ID>.out</code>	The epochs, and the devices of the first step
<code>logs/cx-pytorch-<JOB_ID>.err</code>	Errors and warnings
<code>runs/<JOB_ID>/config.json</code>	Settings, seeds, versions, host and GPU

What the job leaves behind

Every file carries the job's number, so two runs never mix.

<code>logs/cx-pytorch-<JOB_ID>.out</code>	The epochs, and the devices of the first step
<code>logs/cx-pytorch-<JOB_ID>.err</code>	Errors and warnings
<code>runs/<JOB_ID>/config.json</code>	Settings, seeds, versions, host and GPU
<code>runs/<JOB_ID>/metrics.csv</code>	Train and validation loss and accuracy, every epoch

What the job leaves behind

Every file carries the job's number, so two runs never mix.

<code>logs/cx-pytorch-<JOB_ID>.out</code>	The epochs, and the devices of the first step
<code>logs/cx-pytorch-<JOB_ID>.err</code>	Errors and warnings
<code>runs/<JOB_ID>/config.json</code>	Settings, seeds, versions, host and GPU
<code>runs/<JOB_ID>/metrics.csv</code>	Train and validation loss and accuracy, every epoch
<code>runs/<JOB_ID>/summary.json</code>	Written only after a successful finish

What the job leaves behind

Every file carries the job's number, so two runs never mix.

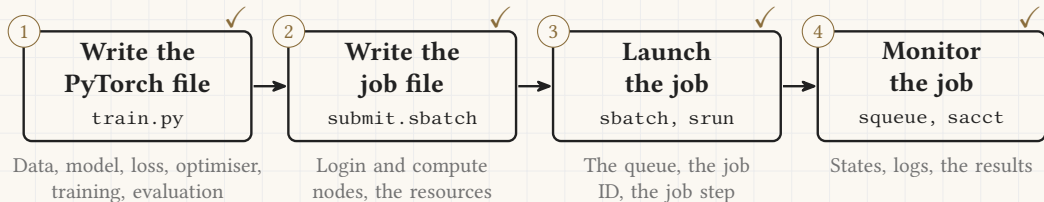
<code>logs/cx-pytorch-<JOB_ID>.out</code>	The epochs, and the devices of the first step
<code>logs/cx-pytorch-<JOB_ID>.err</code>	Errors and warnings
<code>runs/<JOB_ID>/config.json</code>	Settings, seeds, versions, host and GPU
<code>runs/<JOB_ID>/metrics.csv</code>	Train and validation loss and accuracy, every epoch
<code>runs/<JOB_ID>/summary.json</code>	Written only after a successful finish

The `FIRST_STEP` line of the log: where the first step's five tensors were

model cuda:0	features cuda:0	labels cuda:0	logits cuda:0	loss cuda:0
-----------------	--------------------	------------------	------------------	----------------

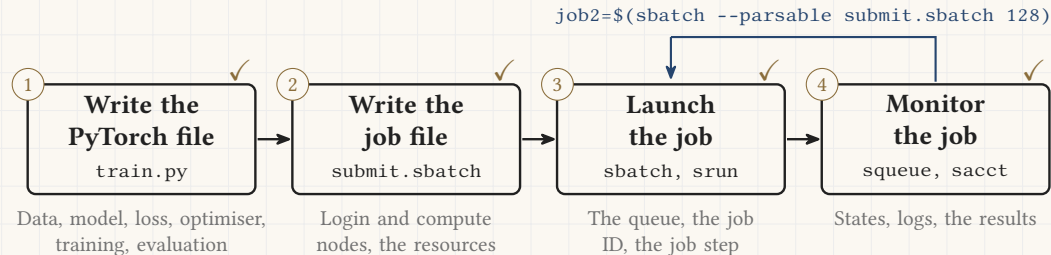
Once more, at width 128

The same file and the same job file, one number changed. Two folders to compare.



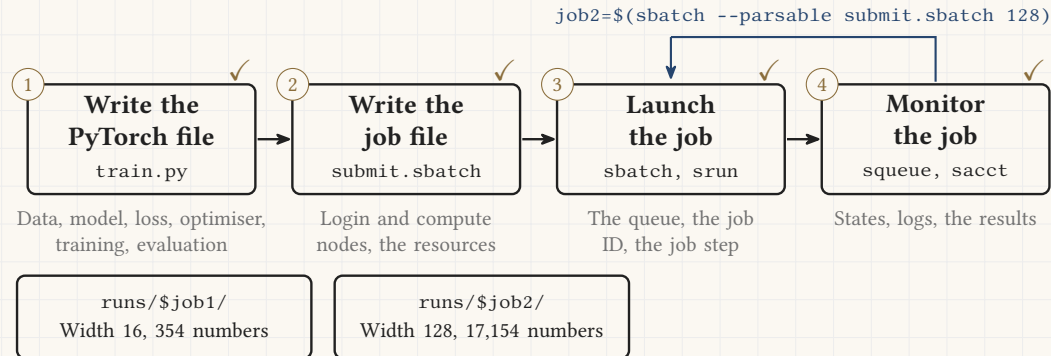
Once more, at width 128

The same file and the same job file, one number changed. Two folders to compare.



Once more, at width 128

The same file and the same job file, one number changed. Two folders to compare.



Once more, at width 128

The same file and the same job file, one number changed. Two folders to compare.

